

UNIVERSIDADE FEDERAL DE SANTA CATARINA - UFSC
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA - INE
INE5645 - PROGRAMAÇÃO PARALELA E DISTRIBUÍDA

RELATÓRIO DO TRABALHO 1

Igor Velmud Bandero
Kalel Gomes de Freitas

FLORIANÓPOLIS - SC
Maio de 2026

Protótipo de Sistema de Vendas Multithread

1. Arquitetura de Software e Padrões de Projeto Adotados

O projeto foi desenvolvido na linguagem C, usamos a API POSIX Threads (Pthreads) para garantir a execução paralela real, com aproveitamento de múltiplos núcleos do processador. O fluxo dos pedidos baseia-se em um modelo de Pipeline, em que blocos independentes processam porções das tarefas simultaneamente. Para a orquestração do *multithreading*, foram utilizados os seguintes padrões de projeto para programação paralela:

1.1. Pipeline (Arquitetura de Fluxo)

Os dados transitam através de quatro estágios de processamento:

- 1) Geração do pedido de cliente
- 2) Validação do cadastro do cliente
- 3) Validação da operação financeira
- 4) Logística de entrega do item.

As transições ocorrem através de filas intermediárias, de forma que nenhum estágio fica impedido de trabalhar caso haja itens a serem processados.

1.2. Produtor-Consumidor

Padrão da implementação das conexões entre os estágios. A inserção em cada estrutura da Fila é realizada por produtores (ex: Clientes gerando compras), e a remoção é realizada pelos consumidores (ex: threads de Cadastro). Mutexes e Variáveis de Condição (“pthread_cond_t”) são usados para suspender as execuções e evitar falhas de concorrência ou desperdício de CPU se uma fila encher (bloqueio do push) ou estiver vazia (bloqueio de pop).

1.3. Thread Pool

Em vez de sobrecarregar o Sistema Operacional com a geração de uma nova *Thread* a cada pedido (o que causaria gargalo de recursos), foram criados pools pré-alocados de trabalho com números fixos de *workers* para cada etapa. Isso permite a estabilidade, com threads aguardando e processando os buffers da fila.

1.4. Monitor

Visto no comportamento da Fila. Todas as variáveis relativas ao *status* da fila (ponteiros de buffer, contadores de lotação e flags de estado do pipeline) estão encapsuladas em funções *_thread-safe_* que garantem o acesso exclusivo pelo uso de Mutex e condições, atuando na prática como um Monitor.

2. Casos de Uso e Testes da Solução

Para testar o sistema foram definidas as seguintes validações e simulações que podem ser acompanhadas durante a execução do programa:

2.1. Cenário A: Execução Padrão e Sem Imprevistos

- Configuração: TAXA_FALHA_CADASTRO = 5, TAXA_FALHA_FINANCEIRO = 10, TAXA_FALHA_LOGISTICA = 5 com 100 pedidos via 3 Produtores simultâneos.

- Saídas Observadas:

Os logs mostram a alternância de geração e processamento:

Cliente 1 gerou pedido 1
Cliente 2 gerou pedido 2
Cadastro 1 recebeu pedido 1
Cadastro 2 recebeu pedido 2
Financeiro 1 recebeu pedido 1 ...

No resumo final, observamos estatísticas da execução: o número "Total gerados" bate com a soma de reprovações e das entregas concluídas, o que mostra que nenhum pedido foi perdido em memória e nem processado duas vezes.

2.2. Cenário B: Taxas de Falha Extremas (Comportamento de Descarte)

- Configuração: Todas as taxas de falha configuradas para 50%.

- Saídas Observadas:

A fila final da logística é acionada por uma quantidade bem menor de instâncias. Isso valida o uso do padrão Produtor-Consumidor com repasse inteligente. Os *workers* despendem recursos bloqueados esperando

aprovações anteriores, demonstrando o efeito gargalo em cadeia, natural a modelos reais. Uma mensagem de log indicando "Financeiro 2 reprovou pedido X" retira o item do contexto do pipeline sem corrupção dos arrays e mantém o alinhamento com as somas de descarte.

2.3. Cenário C: Gargalos Extremos / *Deadlock Validation*

- Configuração: TAM_FILA = 5 (pequena) combinada com total de pedidos massivos (5000) e introdução de lentidão de processamento via função *sleep()* nos workers.
- Saídas Observadas:
O programa executa sem o travamento das threads, que é clássico em falhas do tipo *Deadlock*. O sinal de *Broadcast* (*pthread_cond_broadcast*) adotado na função *fila_close()* é disparado ao constatar exaustão sistemática dos *_publishers_* da linha matriz, destrancando os loopings infinitos de todos os workers e garantindo o encerramento.